



PROGRAMAÇÃO I

Aula 05

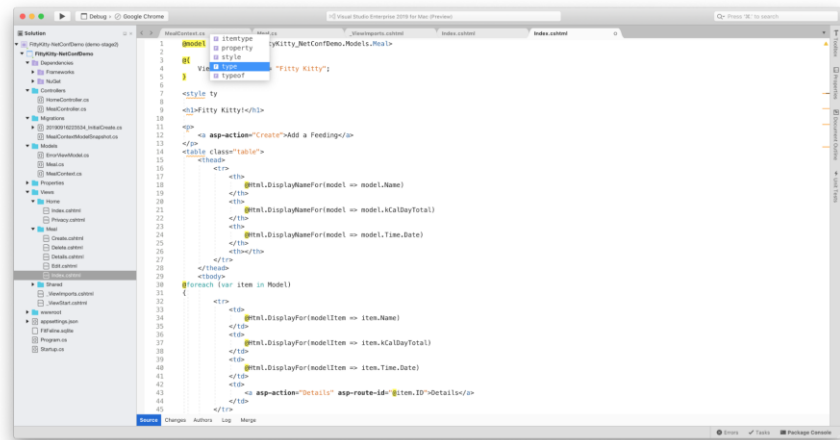
Vinicius Bischoff

IDE - Integrated Development Environment

A teal rectangular graphic with a network of white dots connected by thin white lines, creating a mesh-like background. The text 'IDE' is prominently displayed in large, bold, white capital letters in the center. Below it, the words 'Integrated Development Environment' are written in a smaller, white, sans-serif font.

IDE
Integrated Development Environment

O ambiente de desenvolvimento integrado (IDE do inglês Integrated Development Environment), é uma ferramenta de desenvolvimento para editar o código, acessar um terminal, executar um script , debugar e **compilar** usando um único ambiente.



Ambiente de desenvolvimento integrado (IDE)

Um IDE é um programa repleto de funcionalidades que podem ser usadas por muitos aspectos no desenvolvimento de software, que inclui ferramentas de preenchimento de código, plugins, e muitos outros recursos para facilitar o processo de desenvolvimento de software.

Porém, atualmente existem diversas IDEs disponíveis no mercado, algumas desenvolvidas para linguagens de programação específicas, outros editores de código que se comportam como IDE. Vamos descobrir um pouco mais sobre isso.

Ambiente de desenvolvimento integrado (IDE)

Dev que é dev
usa VSCode



Dev que é dev
usa VIM



Dev que é dev
usa NANO



Dev que é dev
usa IntelliJ



Dev que é dev
usa Visual Studio



Ambiente de desenvolvimento integrado (IDE)

Editor de código e IDE

- Os editores de código podem ser editores de texto simples, como Notepad++ ou VIM, por exemplo, mas que não aprimoram o processo de edição de código.
- Porém, existem alguns editores de código com diversas funcionalidades integradas, plugins e terminais acoplados que facilitam muito o desenvolvimento.

Editor de código e IDE

- Como exemplo podemos citar o Atom, o Sublime Text, e o famoso Visual Studio Code.

Mas qual a diferença entre um editor de código e uma IDE?

Editores de código são editores de texto com poderosos recursos integrados e funcionalidades especializadas para simplificar e acelerar o processo de edição de código.

Editor de código e IDE

IDE, por outro lado, é um conjunto de ferramentas de desenvolvimento de software projetadas para tornar a codificação mais fácil.

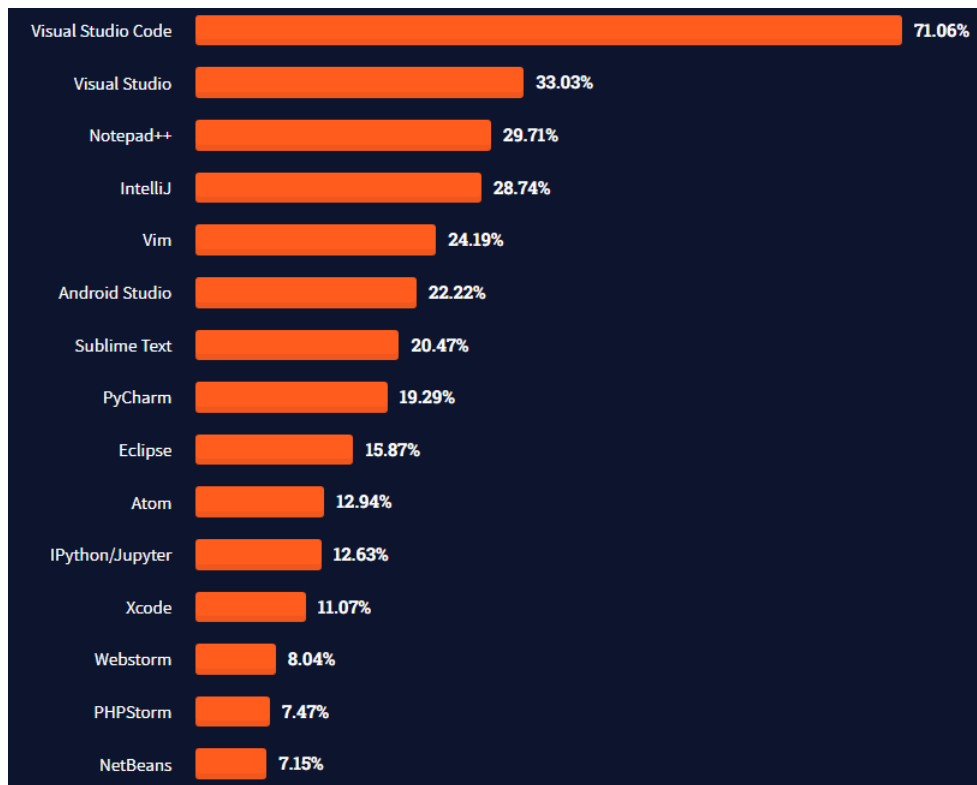
Em outras palavras, um IDE possui um editor de código, um **depurador, compilador e outros recursos** importantes, tudo em uma única ferramenta.

Editor de código e IDE

Existem diferentes tipos de IDEs, alguns de grandes organizações dos quais você provavelmente já ouviu falar, como [Visual Studio da Microsoft](#), [Xcode da Apple](#), e o [Android Studio da Google](#). Os IDEs populares são [Eclipse](#), [IntelliJ IDEA](#), [NetBeans](#), [PyCharm](#), entre outros.

[Neste link](#), você encontra uma lista de outras IDEs relacionadas com suas respectivas linguagens.

IDE e editor de código mais utilizados



Segundo o [Stack Overflow](#), os ambientes de desenvolvimento mais utilizados por aqueles de desenvolvem são Visual Studio Code, Visual Studio e IntelliJ.

Ambiente de desenvolvimento integrado (IDE)

Opinião pessoal

“Para aqueles que programam em Java, IntelliJ, NetBeans ou Eclipse são excelentes recomendações.

Já para quem desenvolve utilizando o Javascript, Visual Studio Code e Atom.

A escolha do editor ou IDE é pessoal e depende muito da linguagem ou área de atuação. Além disso, existem pessoas que se adaptam melhor com um ambiente do que outro.”



Desenvolvendo aplicações Java com o VS Code

Ambiente de desenvolvimento integrado (IDE)



Visual Studio Code



Ambiente de desenvolvimento integrado (IDE)

O VS Code é um editor de código-fonte autônomo e multiplataforma, que pode ser utilizado no Windows, macOS e Linux.

O grande diferencial desse editor é que além de ter o conceito “*lightweight*”, ou seja, de ser um ambiente leve em comparação a outras IDEs, ele também é altamente customizável, com centenas de extensões existentes para serem utilizadas.

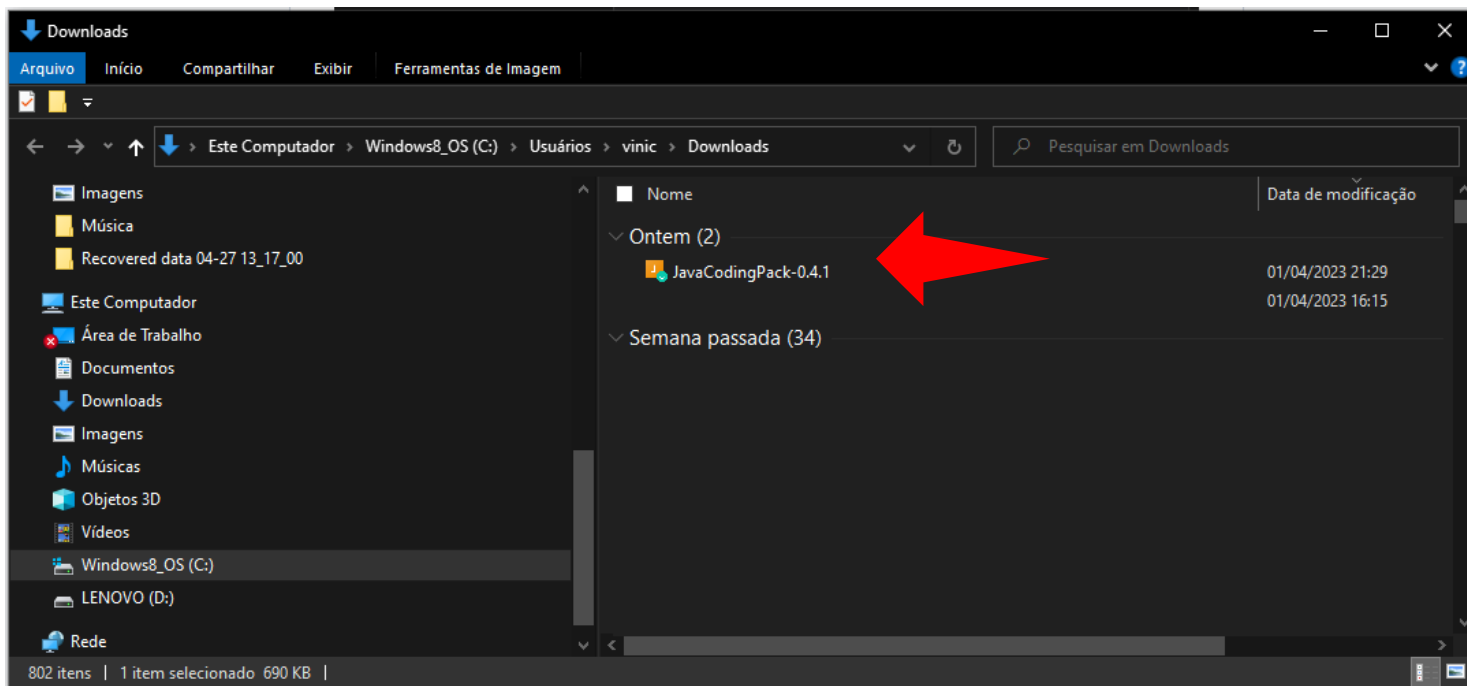
Instalação

Se o seu sistema operacional é Windows ou o macOS, o VS Code já tem um pacote de instalação que vai prover os principais recursos para o *setup* inicial do ambiente, que é o [Coding Pack for Java](#).

Nesse pacote de instalação já estão incluídos os seguintes itens:

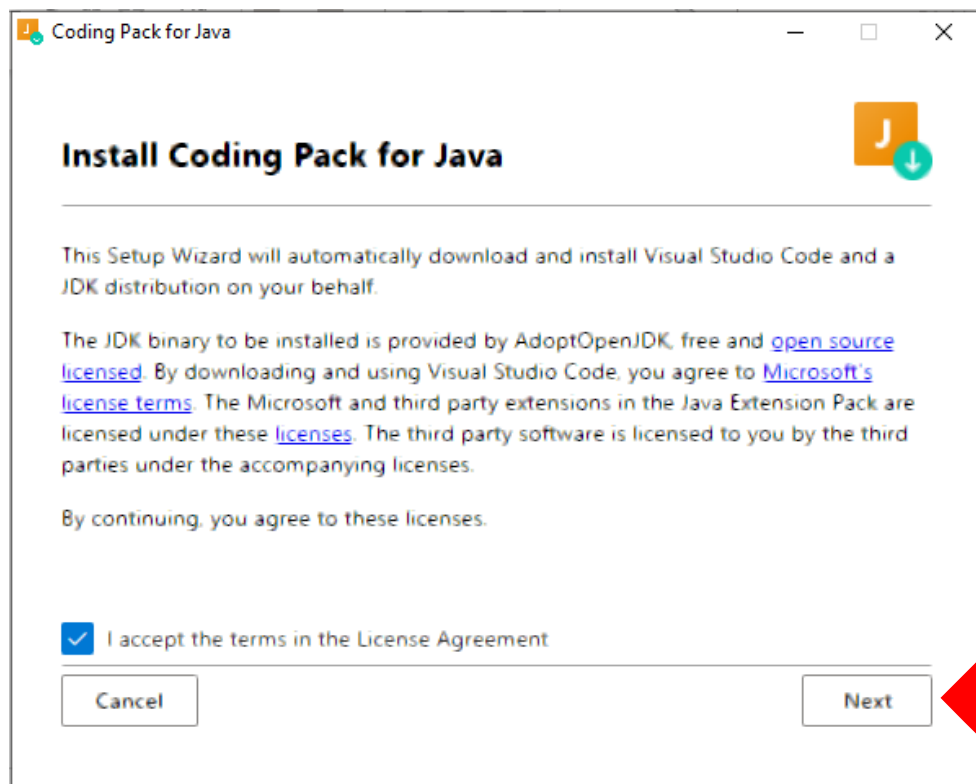
- **JDK (*Java Development Kit* - pode variar a versão, de acordo com a data que você fizer o download, mas geralmente é uma versão LTS, ou seja, com suporte de longo prazo. Nesta aula a versão é a 17);**
- **VS Code**
- **Extensões essenciais Java (ou *Java Extension Pack*)**

Instalação



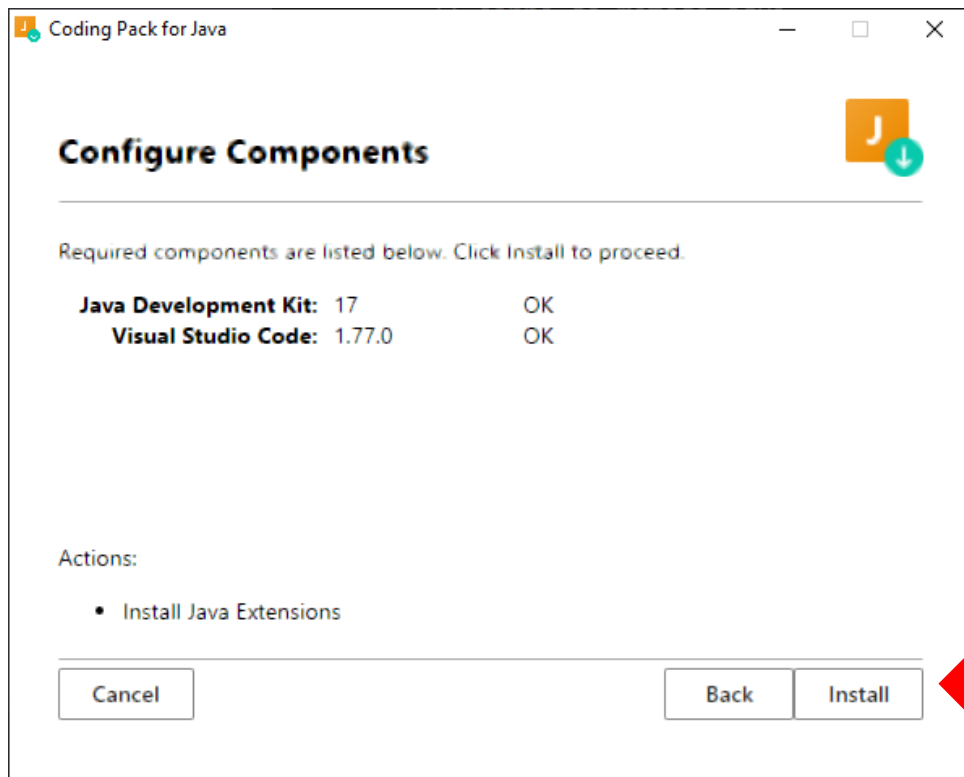
Ambiente de desenvolvimento integrado (IDE)

Instalação



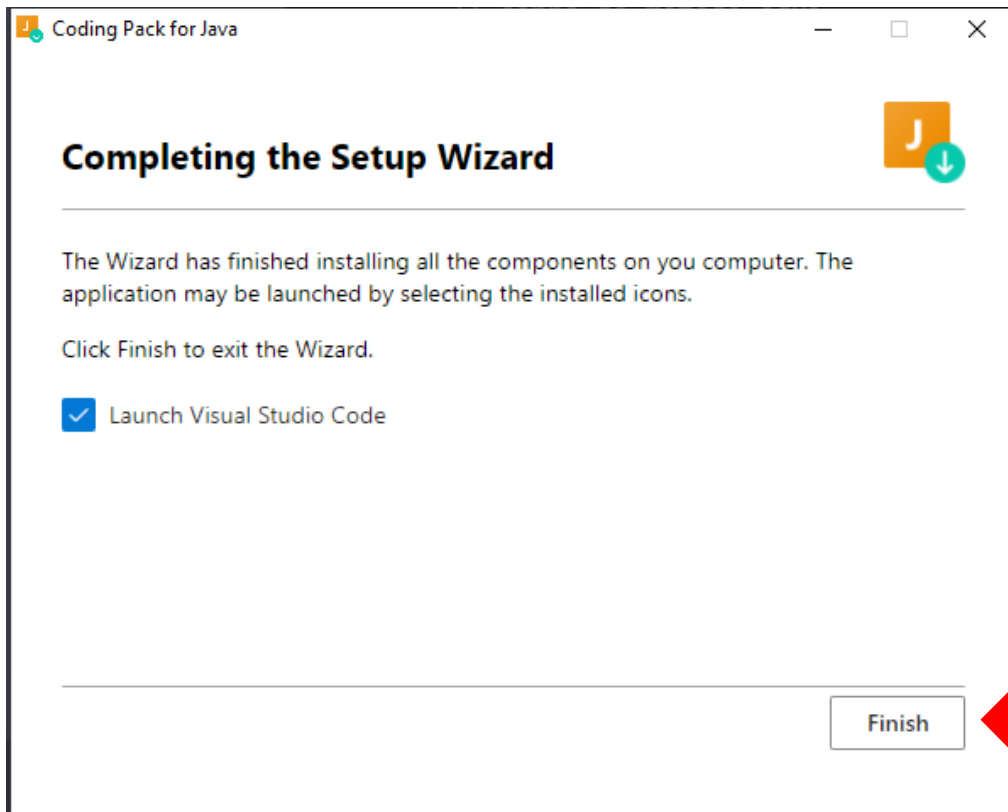
Ambiente de desenvolvimento integrado (IDE)

Instalação



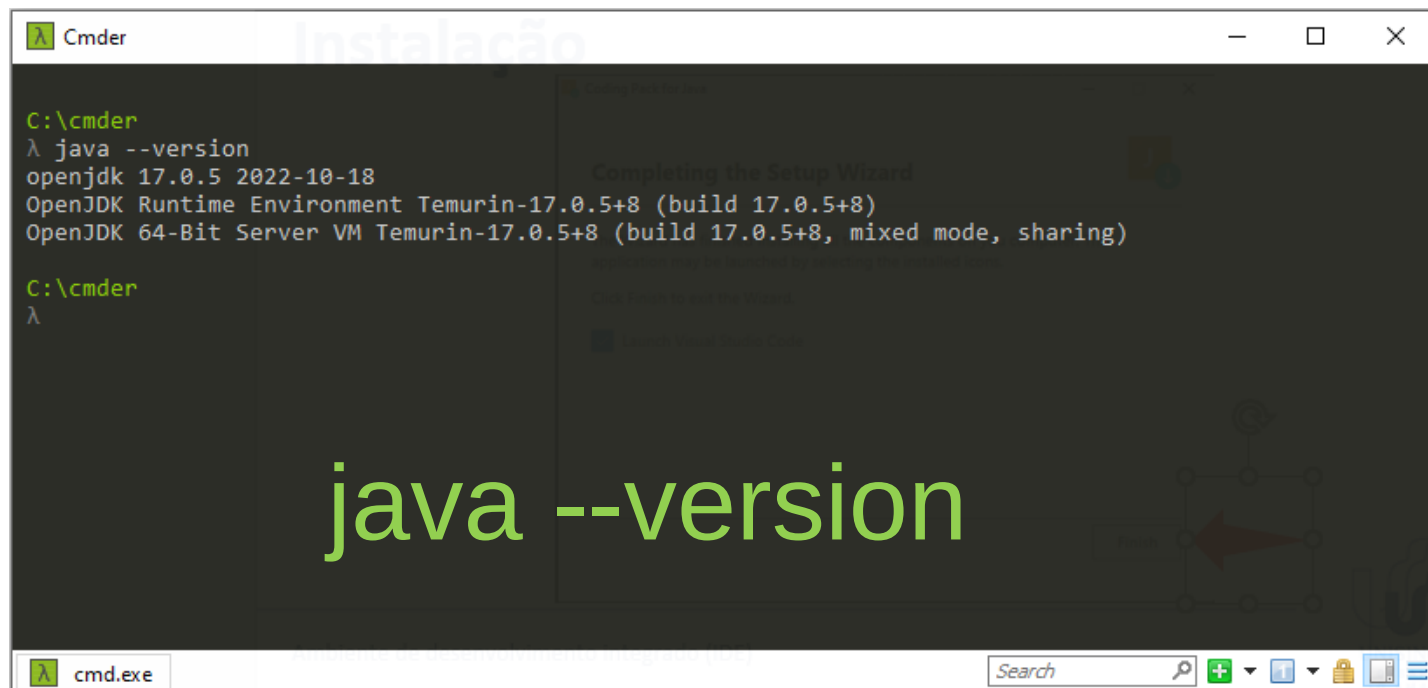
Ambiente de desenvolvimento integrado (IDE)

Instalação



Ambiente de desenvolvimento integrado (IDE)

Instalação



```
C:\cmder
λ java --version
openjdk 17.0.5 2022-10-18
OpenJDK Runtime Environment Temurin-17.0.5+8 (build 17.0.5+8)
OpenJDK 64-Bit Server VM Temurin-17.0.5+8 (build 17.0.5+8, mixed mode, sharing)

C:\cmder
λ
```

java --version

Ambiente de desenvolvimento integrado (IDE)

O que são essas extensões essenciais Java?

O **Java Extension Pack** é um pacote que contempla seis extensões essenciais para facilitar o desenvolvimento, *build*, *debug* e teste de aplicações Java. São essas:

Ambiente de desenvolvimento integrado (IDE)

Language Support for Java:

Essa extensão é fornecida pela própria Red Hat e é responsável por diversos recursos que otimizam o desenvolvimento, como: *code completion*, *as-you-type reporting* (que já detecta e destaca erros de sintaxe e *compilação* no momento da digitação), organização dos *imports*, ações de código (como gerar *getters* e *setters*, construtores, *toString*), trechos de códigos prontos (*code snippets*), suporte a arquivos individuais com extensão .java, para que sejam executados no próprio terminal do VS Code e muitos outros recursos já conhecidos por quem já utiliza outras IDEs.

Debugger for Java

É uma [extensão da Microsoft](#) que vai permitir que você faça a depuração do código, incluindo *breakpoints* (pontos de interrupção) para avaliar valores de variáveis e o que está acontecendo na aplicação, tendo a possibilidade de fazer substituições ‘a quente’ no código (*hot code replaces*). Tem recursos de *step in/out/over* para navegar nos métodos, pausa e continue, além de outras facilidades que ajudam na investigação de problemas e comportamentos inesperados na aplicação.

Test Runner for Java

Essa [extensão](#), também da Microsoft, possibilita a depuração e execução de casos de teste escritos em Java, com uso dos frameworks JUnit ou TestNG. Essa extensão tem como pré-requisito a instalação das duas primeiras citadas.

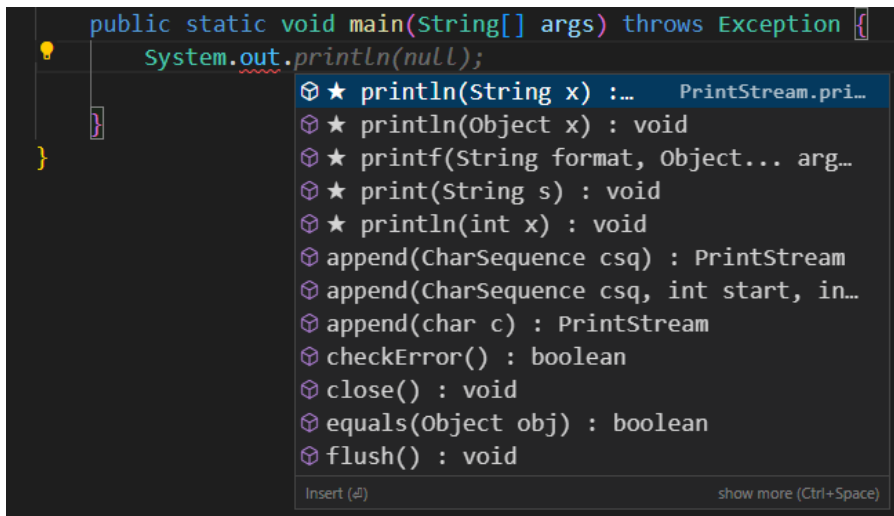
Maven for Java

A extensão do Maven fornece suporte para criação de projetos seguindo algum arquétipo do **Maven**, bem como atalhos para inclusão e controle das dependências no POM (*Project Object Model*) além da customização de comandos.

Project Manager for Java

Essa extensão atua como um gerenciador de projetos, oferecendo algumas facilidades no que tange à visualização das principais configurações e estrutura do projeto aberto no VS Code.

Visual Studio IntelliCode



Por fim, a extensão [IntelliCode](#) que é uma espécie de *intellisense* assistido, que já coloca como atalho no topo da lista do code completion, os comandos que tem mais possibilidades de serem utilizados, conforme o exemplo abaixo, onde após a digitação das palavras *System.out* ele já sugere os comandos mais usados, como *println*, *printf* e assim sucessivamente:

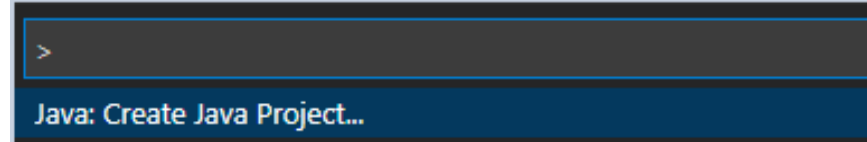
Vamos abrir o VS Code e começar a desenvolver?

Para executar a maioria dos comandos, utilizamos o atalho:

Ctrl + Shift + P

ou pelo

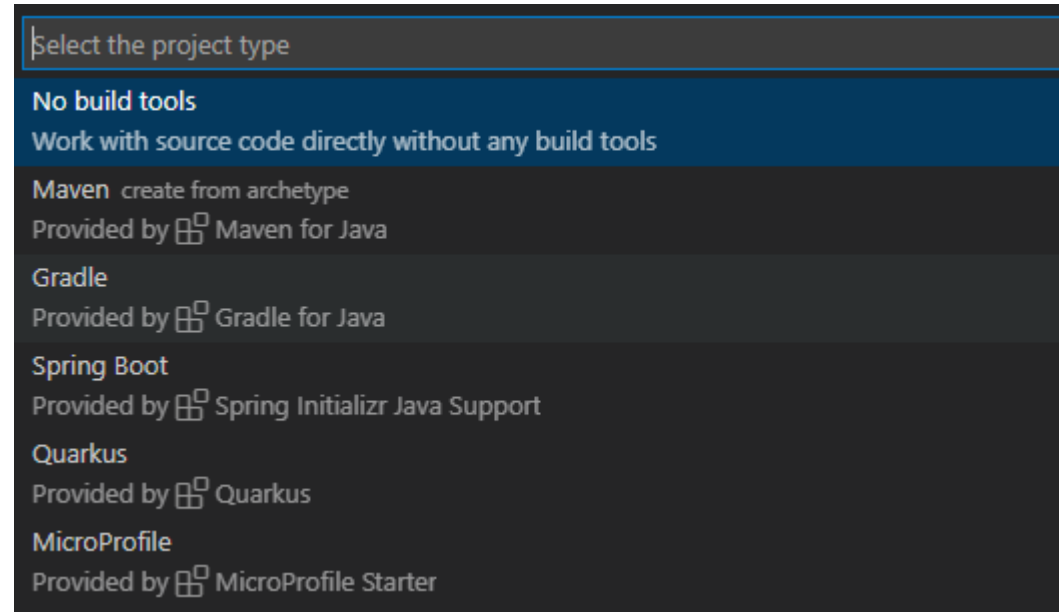
menu *View* → *Command Palette*.



O nome do menu pode mudar caso esteja sendo usado em outro idioma. Aqui vamos manter as dicas utilizando o inglês por padrão. Para iniciar um novo projeto Java, vamos abrir a paleta de comando e escolher a opção *Java: Create Java Project*.

Repare que ele já abre uma nova tela com várias opções de projeto Java.

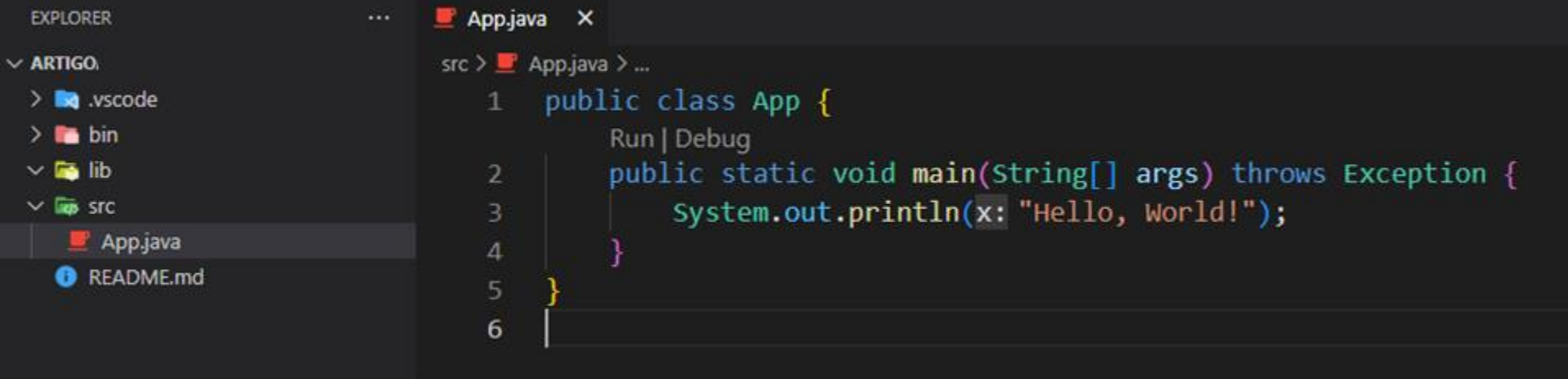
O VS Code tem extensão para facilitar a criação com qualquer que seja a opção escolhida: *Maven*, *Gradle*, *Spring Boot*, *Quarkus* ou *MicroProfile*.



Vamos escolher a opção **“No build tools”** apenas para ver algumas opções básicas do VS Code.

Deverá ser informada a pasta na qual o projeto será gerado e o nome do mesmo.

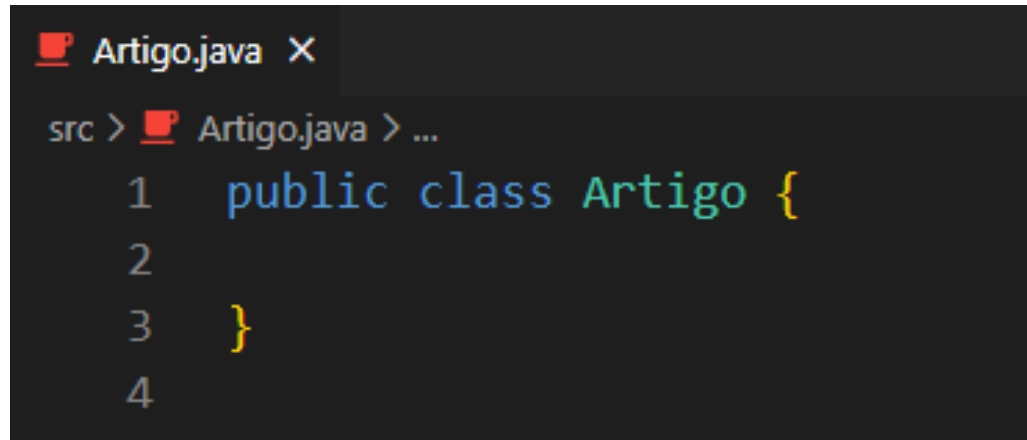
Ao prosseguir, já será criada uma estrutura de pacotes, com as pastas *.vscode* (que contém o arquivo *settings.json*), a pasta *bin* (que vai conter os arquivos com a extensão *.class*), a pasta *lib* (onde ficam os recursos ou bibliotecas utilizadas na aplicação) e a pasta *src* (onde fica o código-fonte, no nosso caso, as classes).



The image shows a screenshot of the Visual Studio Code (VS Code) integrated development environment (IDE). On the left, the 'EXPLORER' sidebar displays the project structure under the name 'ARTIGO'. It includes folders for '.vscode', 'bin', 'lib', and 'src'. The 'src' folder is expanded, showing the file 'App.java' selected, along with a 'README.md' file. The main editor window displays the code for 'App.java'. The code defines a public class 'App' with a 'main' method that prints 'Hello, World!' to the console. The code is as follows:

```
1 public class App {  
    Run | Debug  
2     public static void main(String[] args) throws Exception {  
3         System.out.println(x: "Hello, World!");  
4     }  
5 }  
6
```

Já é criada a primeira classe, a **App.java**, com o método *main* e o nosso famoso **Hello, World!**.



```
Artigo.java X
src > Artigo.java > ...
1  public class Artigo {
2
3  }
4
```

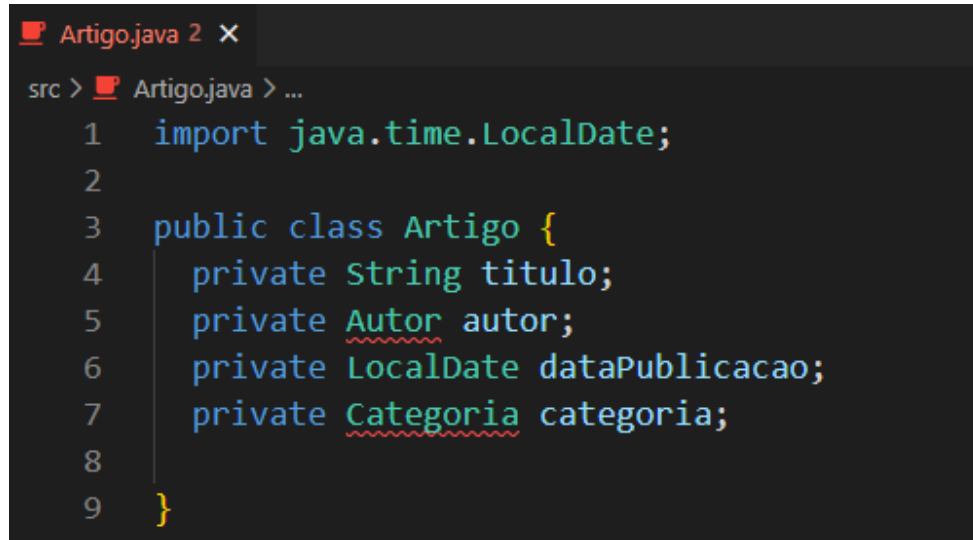
Para criar uma nova classe, podemos utilizar novamente a paleta de comandos e buscar a opção *Java: New Java Class*. Vou dar o nome Artigo para prosseguir com a criação da classe **Artigo.java**.

Vamos incluir alguns atributos e usar os recursos da IDE para finalizar as implementações dos *getters*, *setters*, construtores e *toString*.

```
Artigo.java 2 x
src > Artigo.java > ...
1  import java.time.LocalDate;
2
3  public class Artigo {
4      private String titulo;
5      private Autor autor;
6      private LocalDate dataPublicacao;
7      private Categoria categoria;
8
9  }
```

Veja que aqui na imagem a seguir, após a inclusão de alguns atributos, alguns já aparecem em vermelho.

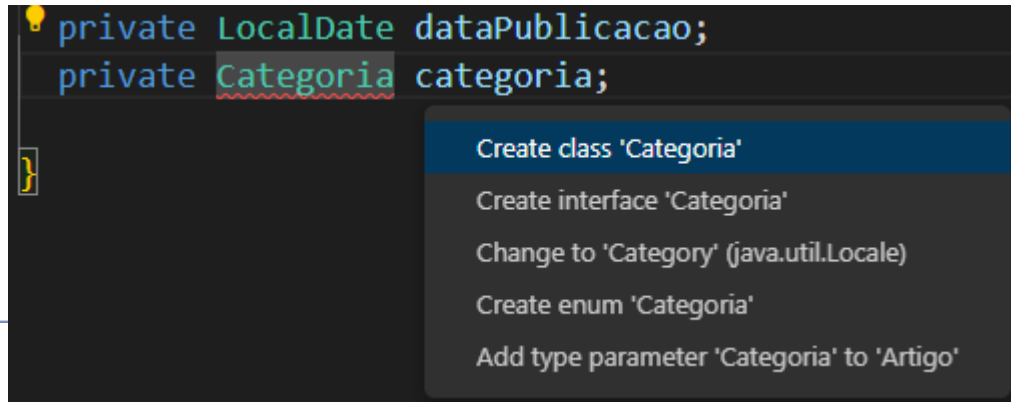
Veja também que para o tipo **LocalDate** já foi feito o import automático na linha um, indicando o nome completo (pacote + classe), que é `java.time.LocalDate`.



```
Artigo.java 2 X
src > Artigo.java > ...
1  import java.time.LocalDate;
2
3  public class Artigo {
4      private String titulo;
5      private Autor autor;
6      private LocalDate dataPublicacao;
7      private Categoria categoria;
8
9  }
```

Ambiente de desenvolvimento integrado (IDE)

Para os tipos que estão sublinhados de vermelho, ao clicar no ícone da lâmpada ou clicar com o botão direito no `ter`, ou ainda, usar o atalho **Ctrl + .**, ele já faz algumas sugestões, sobre criar uma nova classe, interface, enum ou até mesmo usar como referência a classe *Category* do pacote **java.util.Locale**, conforme a imagem abaixo:

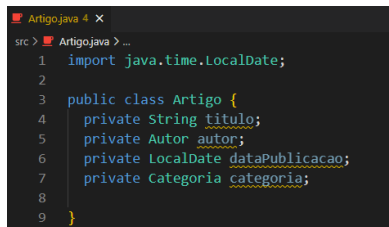


Poderíamos escolher criar um *enum* para Categoria e uma nova classe para Autor e continuar codificando nossa aplicação. Uma vez conhecidos os tipos, ele passa a apresentar um **sublinhado amarelo**, indicando que o valor do nosso atributo não está sendo usado em lugar nenhum

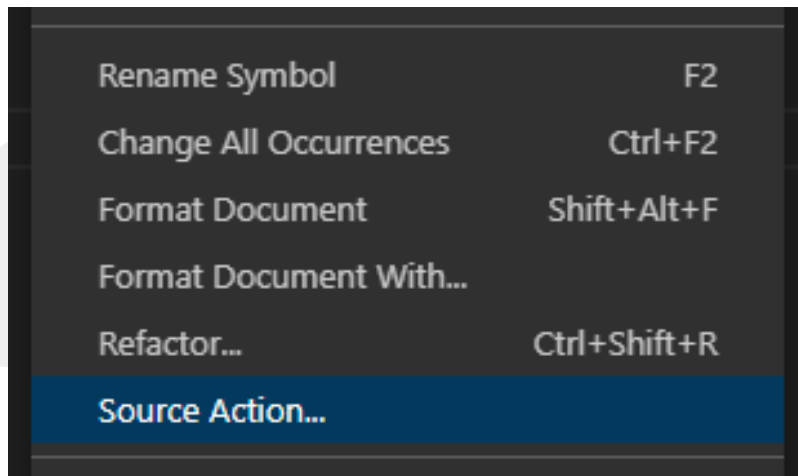
```
Artigo.java 4 X
src > Artigo.java > ...
1  import java.time.LocalDate;
2
3  public class Artigo {
4      private String titulo;
5      private Autor autor;
6      private LocalDate dataPublicacao;
7      private Categoria categoria;
8
9  }
```

Para isso, vamos usar os atalhos do editor para criar automaticamente os *getters* e *setters*.

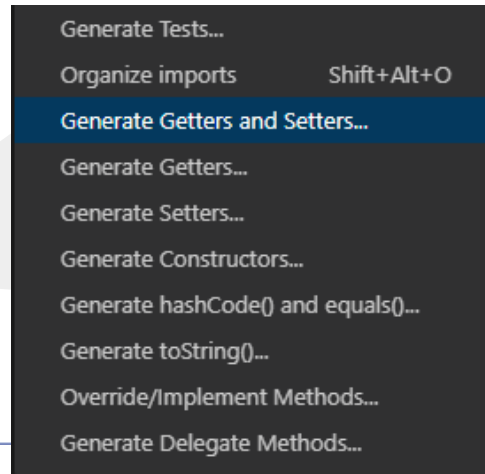
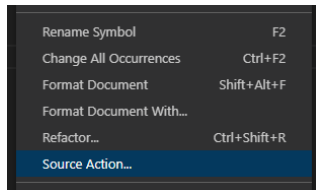
Para isso, basta clicar com o botão direito na janela do editor, onde está aberta a sua classe e navegar até a opção **Source Action**.



```
Artigo.java 4 X
src > Artigo.java > ...
1  import java.time.LocalDate;
2
3  public class Artigo {
4      private String titulo;
5      private Autor autor;
6      private LocalDate dataPublicacao;
7      private Categoria categoria;
8
9  }
```

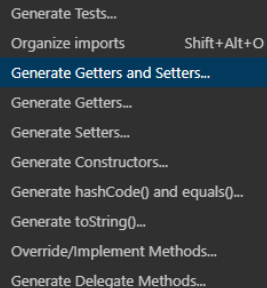


Algumas opções vão aparecer, como a de gerar testes, organizar os *imports* (incluindo referências ou removendo referências não utilizadas), gerar somente os *getters*, somente os *setters*, gerar ambos simultaneamente, construtores, e outras opções, como demonstrado na imagem.

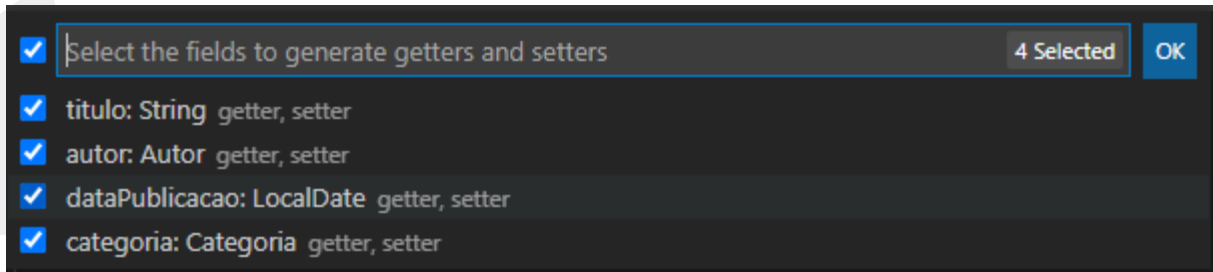


Ambiente de desenvolvimento integrado (IDE)

Vamos gerar primeiramente os *getters* e *setters*:



Generate Tests...
Organize imports Shift+Alt+O
Generate Getters and Setters...
Generate Getters...
Generate Setters...
Generate Constructors...
Generate hashCode() and equals()...
Generate toString()...
Override/Implement Methods...
Generate Delegate Methods...



É possível escolher todos ou somente alguns atributos.
Ao escolher todos, podemos observar que ele já vai criar corretamente os métodos de forma automática.

Vamos gerar primeiramente os *getters* e *setters*:

```
public String getTitulo() {  
    return titulo;  
}  
  
public void setTitulo(String titulo) {  
    this.titulo = titulo;  
}  
  
public Autor getAutor() {  
    return autor;  
}  
  
public void setAutor(Autor autor) {  
    this.autor = autor;  
}  
  
public LocalDate getDataPublicacao() {  
    return dataPublicacao;  
}  
  
public void setDataPublicacao(LocalDate dataPublicacao) {  
    this.dataPublicacao = dataPublicacao;  
}  
  
public Categoria getCategoria() {  
    return categoria;  
}  
  
public void setCategoria(Categoria categoria) {  
    this.categoria = categoria;  
}
```

Podemos fazer o mesmo procedimento para criar um construtor com todos os atributos e também um override do método toString.

Clicar com o botão direito na janela >> Source Action
>> **Generate Constructors...**

```
public Artigo(String titulo, Autor autor, LocalDate dataPublicacao, Categoria categoria) {  
    this.titulo = titulo;  
    this.autor = autor;  
    this.dataPublicacao = dataPublicacao;  
    this.categoria = categoria;  
}
```

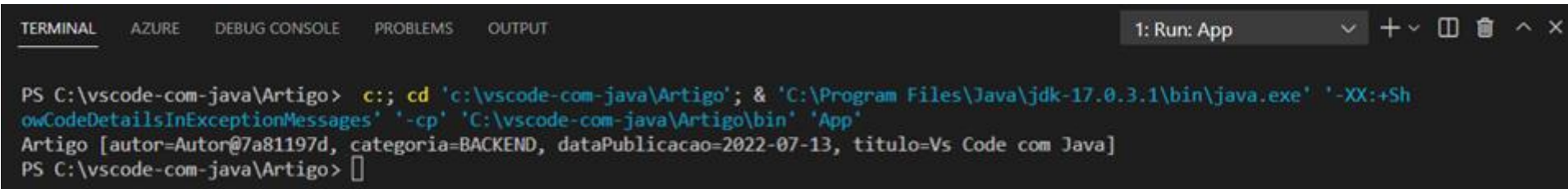
A mesma coisa para o toString. Botão direito na janela >>
Source Action >> **Generate toString()**...

```
@Override
public String toString() {
    return "Artigo [autor=" + autor + ", categoria=" + categoria +
        ", dataPublicacao=" + dataPublicacao + ", titulo="
        + titulo + "]";
}
```

Para finalizar, vamos criar um objeto da classe Artigo na nossa classe **App** e visualizá-lo no Terminal.
Na classe App.java, vou fazer a seguinte implementação:

```
App.java ×
src > App.java > ...
1  import java.time.LocalDate;
2
3  public class App {
4      Run | Debug
      public static void main(String[] args) throws Exception {
5          var artigo = new Artigo(titulo: "Vs Code com Java", new Autor(nome: "Jacqueline Oliveira"),
6                                  LocalDate.of(year: 2022, month: 07, dayOfMonth: 13), Categoria.BACKEND);
7          System.out.println(artigo);
8      }
9  }
```

Ao clicar no atalho *Run*, logo acima do método main, vamos ter a seguinte saída no terminal:



The image shows a screenshot of a Visual Studio Code terminal window. The terminal has tabs for 'TERMINAL', 'AZURE', 'DEBUG CONSOLE', 'PROBLEMS', and 'OUTPUT'. The 'TERMINAL' tab is active. In the top right corner of the terminal, there is a dropdown menu showing '1: Run: App'. The terminal content shows a PowerShell prompt 'PS C:\vscode-com-java\Artigo>' followed by a command to run a Java application with specific JVM options. The output shows the application running and printing some metadata.

```
PS C:\vscode-com-java\Artigo> c:; cd 'c:\vscode-com-java\Artigo'; & 'C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\vscode-com-java\Artigo\bin' 'App'
Artigo [autor=Autor@7a81197d, categoria=BACKEND, dataPublicacao=2022-07-13, titulo=Vs Code com Java]
PS C:\vscode-com-java\Artigo>
```

Ambiente de desenvolvimento integrado (IDE)

Java: o que é, linguagem



```
class Java {  
    public static void main (String args[]) {  
        System.out.println("Java: Um guia para iniciar nessa tecnologia");  
    }  
}
```

Pesquisa StackOverflow



O Java é uma das **plataformas** mais utilizadas para o **desenvolvimento de aplicações** ao redor do mundo. Além da plataforma, segundo dados da [Pesquisa “Tecnologias mais populares de 2022”](#), realizada pelo StackOverflow, a linguagem Java figura como a 6ª mais utilizada por pessoas desenvolvedoras, correspondendo a 33% do total pesquisado.

O que é Java?

O Java, como **plataforma de programação**, nasceu no ano de 1995 dentro dos laboratórios da empresa Sun Microsystems como resultado de uma extensa pesquisa científica e tecnológica. A plataforma Java entrega um **ambiente completo** para o **desenvolvimento e execução de programas**, sendo composta por:

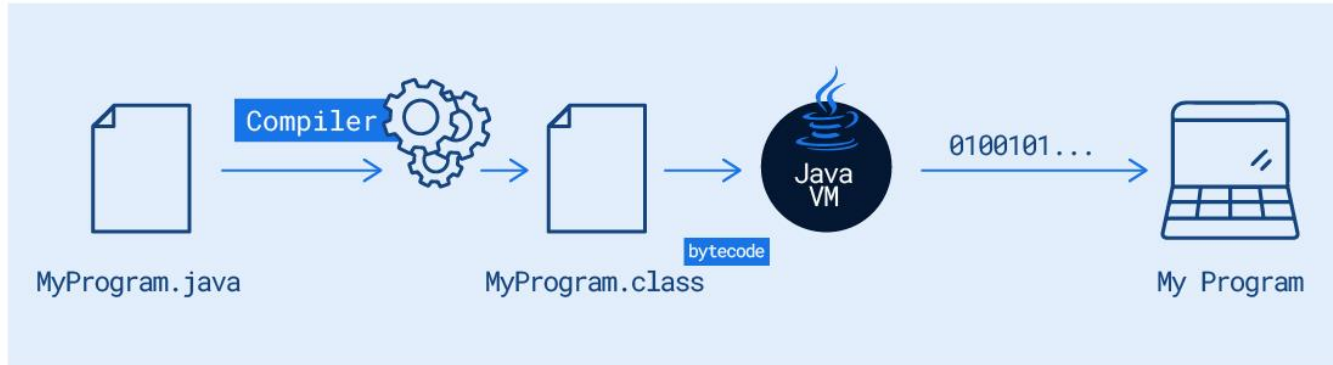
O que é Java?

- Uma **linguagem de programação de alto nível orientada a objetos**;
- Máquina Virtual (*Java Virtual Machine* ou JVM), que garante **independência de plataforma**, pois o código executa na máquina virtual e essa pode ser portada para outras plataformas como Windows ou Linux;
- *Java Runtime Environment* ou **JRE**, que agrega a máquina virtual e alguns recursos para a execução de aplicações Java; e
- *Java Development Kit* ou **JDK**, que é um conjunto de utilitários que oferece suporte ao desenvolvimento de aplicações.

O que é Java?

No Java, os programas são escritos em um arquivo com a extensão **.java**, que em um processo posterior serão compilados para arquivos com a extensão **.class**. Esses, por sua vez, contêm os códigos a serem executados na máquina virtual, os **bytecodes**.

.java, .class, bytecodes



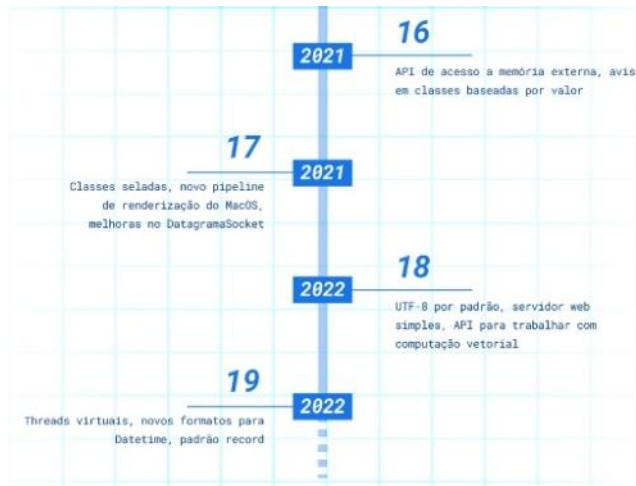
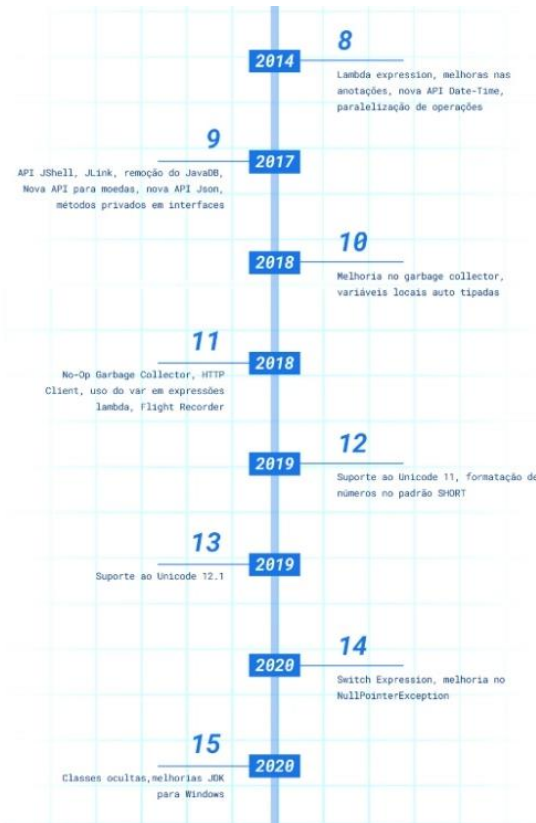
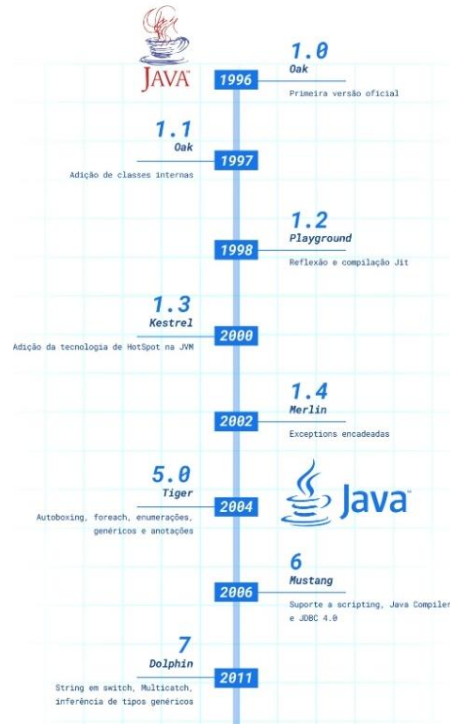
O que é Java?

A **JVM** está **disponível para a maioria dos sistemas operacionais** do mercado, sendo assim conseguimos rodar a mesma aplicação Java no Windows, macOS, Linux, Solaris, entre outros.

Essa funcionalidade implementa um dos conceitos que nasceram forte com o Java:

“escreva uma vez e execute em qualquer lugar!”

Linha do tempo com as versões no Java no decorrer dos anos desde o seu lançamento:



Vale a pena destacar também que o Java, em sua **evolução**, parou de nomear as versões a partir do **Java 8**, e seu **versionamento atualizado é liberado de 6 em 6 meses**, adotando as formas *Preview* e *LTS (Long Time Support)*.

Tecnologia Java

A plataforma Java é estruturada em um portfólio de produtos para desenvolvimento e execução de aplicações, idealizando que um mesmo **programa** possa **funcionar em diferentes sistemas operacionais e dispositivos**.

Atualmente a plataforma está dividida em duas grandes áreas:

- *Java Standard Edition* ou **JavaSE**
- *Java Enterprise Edition* ou **JavaEE**

Java Enterprise Edition ou JavaSE

Componente padrão do Java que fornece um **ambiente** para o **desenvolvimento de aplicações de pequeno e médio porte**, além de um conjunto de APIs base da plataforma e a JVM padrão.

Java Enterprise Edition ou JavaEE

Componente baseado no **JavaSE**,
é **focado no desenvolvimento de aplicações empresariais multicamadas de grande porte** e provê serviços adicionais, ferramentas e APIs para simplificar a criação de aplicações complexas.

O Java é gratuito?

Desde o seu lançamento oficial em 1996 até as suas mais recentes versões, o Java sofreu **evoluções** e **melhorias** que o permitiram se manter como uma plataforma e linguagem competitiva, o que explica sua grande popularidade.

O Java é gratuito?

Mas uma dúvida muito comum, principalmente para quem está iniciando, é a seguinte: **o Java é gratuito?**

Uma pergunta pertinente, uma vez que a gigante de tecnologia Oracle comprou a Sun e a plataforma Java.

O Java é gratuito?

A resposta para essa pergunta encontramos no site da Oracle. **O Java é gratuito para estudo e testes, porém, para uso comercial e suporte** você deverá desembolsar um **valor para licenciamentos.**

Kit de Ferramentas Java: OpenJDK

Hoje existe também uma versão totalmente gratuita do Java e de suas ferramentas que é mantida pela comunidade, contando também com o apoio da Oracle. Trata-se do OpenJDK, que é o **kit de ferramentas** para o desenvolvimento Java. Ele existe desde 2006, porém, desde a compra do Java pela Oracle, o kit passou a ganhar ainda mais força.

Qual a diferença entre o Java Oracle e o OpenJDK?

O Java Oracle é desenvolvido com base no código oficial do projeto Open e permite uso completo e comercial, sendo **diferenciado pelo suporte e a forma de licenciamento**, mas tecnicamente ambas as versões são o mesmo Java.

Para saber mais sobre os custos do Java Oracle acesse [a página da Oracle](#).

Como utilizar a Plataforma Java

O Java como **plataforma de computação** é muito utilizado, pois existe um grande número de aplicações para computador, sites e aplicativos que dependem do Java para funcionar.

Por isso, é muito comum a pergunta: *“Eu preciso do Java no meu computador?”*.

Como utilizar a Plataforma Java

Para executar as **aplicações Java** no nosso computador é necessária a **instalação** do ambiente de execução do Java ou JRE. Ela é uma camada de aplicação e é instalada no sistema operacional, **forneendo a biblioteca de classes** e os **recursos necessários** para a execução do código Java pela JVM.

Como utilizar a Plataforma Java

Como exemplo de aplicações que precisam do Java para funcionar, temos o aplicativo do Imposto de Renda de Pessoa Física (IRPF), utilizado pelo Ministério da Fazenda do Brasil.

Aliás, muitos aplicativos oficiais do governo brasileiro foram desenvolvidos com o Java, como os utilizados para a Declaração do Imposto de Renda Retido na Fonte (DIRF) e a Relação Anual de Informações Sociais (GDRAIS).

Para **configurar o Java** em seu computador, executar aplicativos e componentes criados em Java, você pode **baixar a JRE** na [página oficial da plataforma](#) e efetuar a instalação para o sistema operacional de sua escolha.

• **Visualização da instalação no Windows:**

Windows Which download should I choose?

	Windows Online filesize: 2.16 MB	Instructions	After installing Java, you may need to restart your browser in order to enable Java in your browser.
	Windows Offline filesize: 72.73 MB	Instructions	
	Windows Offline (64-bit) filesize: 83.46 MB	Instructions	

If you use 32-bit and 64-bit browsers interchangeably, you will need to install both 32-bit and 64-bit Java in order to have the Java plug-in for both browsers. » [FAQ about 64-bit Java for Windows](#)

Mac OS X Mac FAQ

	Mac OS X (10.7.3 version and above) filesize: 84.46 MB	Instructions	After installing Java, you may need to restart your browser in order to enable Java in your browser.
--	---	--------------	--

* Oracle Java (Version 7 and later versions) requires an Intel-based Mac running Mac OS X 10.7.3 (Lion) or later and administrator privileges for installation. » [More information](#)

Plataforma Java

- Neste processo de instalação e configuração do Java em seu computador, você pode optar também pela instalação da versão Open:

← → ↺ 🏠 jdk.java.net/19/

jdk.java.net

GA Releases
JDK 19
JDK 18
JHC 8

Early-Access Releases
JDK 20
Jextract
Loom
Metropolis
Panama
Valhalla

Reference Implementations
Java SE 19
Java SE 18
Java SE 17
Java SE 16
Java SE 15
Java SE 14
Java SE 13
Java SE 12
Java SE 11
Java SE 10
Java SE 9
Java SE 8
Java SE 7

Feedback
Report a bug
Archive

OpenJDK JDK 19 General-Availability Release

This page provides production-ready open-source builds of the Java Development Kit, version 19, an implementation of the Java SE 19 Platform under the GNU General Public License, version 2, with the Classpath Exception.

Commercial builds of JDK 19 from Oracle, under a non-open-source license, can be found at the Oracle Technology Network.

Documentation

- Features
- Release notes
- API Javadoc

Builds

Linux/AArch64	tar.gz (sha256)	194401252 bytes
Linux/x64	tar.gz (sha256)	195673378
macOS/AArch64	tar.gz (sha256)	190416515
macOS/x64	tar.gz (sha256)	192323254
Windows/x64	zip (sha256)	194357214

Notes

- If you have difficulty downloading any of these files please contact download-help@openjdk.org.

Feedback

If you have suggestions or encounter bugs, please submit them using the usual Java SE bug-reporting channel. Be sure to include complete version information from the output of the `java --version` command.

International use restrictions

Due to limited intellectual property protection and enforcement in certain countries, the source code may only be distributed to an authorized list of countries. You will not be able to access the source code if you are downloading from a country that is not on this list. We are continuously reviewing this list for addition of other countries.

ORACLE

© 2022 Oracle Corporation and/or its affiliates
Terms of Use - Privacy - Trademarks

Plataforma Java

Obrigado.

